

MongoDB Data Types

A Structured Tutorial Summary

This tutorial explains all major MongoDB data types, how they are stored within documents, and how different formats behave inside collections. It also covers ISODate formatting, time zones, integer ranges, and practical insertion commands.

1. Understanding MongoDB Document Structure

MongoDB stores data in a flexible JSON-like format known as **BSON**. A database contains **collections**, and each collection stores **documents**. A document holds key-value pairs, where each value can use different MongoDB data types.

2. Overview of Common MongoDB Data Types

The tutorial demonstrates a sample user document containing multiple data types:

```
{
  "_id": ObjectId("..."),
  "name": "John",
  "age": 30,
  "married": false,
  "dob": ISODate("2000-10-25T08:00:00Z"),
  "weight": 72.5,
  "kids": null,
  "hobbies": ["music", "sports"],
  "address": {
    "street": "MG Road",
    "city": "Delhi",
    "zip": 110001
  }
}
```

Below are the data types used.

2.1 ObjectId

- Auto-generated unique identifier for every document.
- Assigned automatically when inserting data.

Example:

```
"_id": ObjectId("652fa86943ec...")
```

2.2 String

- Any value in quotes (single or double) becomes a string.
- Even numeric characters become a string if placed in quotes.

Example:

```
"name": "John"
```

2.3 Integer (32-bit and 64-bit)

MongoDB supports two kinds of integers:

32-bit Integer (int32)

- Range: approx. -2,147,483,648 to +2,147,483,647
- Uses 4 bytes

64-bit Integer (int64)

- Supports extremely large values
- Uses 8 bytes

MongoDB automatically determines which integer type to allocate based on value size.

Example:

```
"age": 25
```

2.4 Boolean

- Used to represent `true` or `false` without quotes.

Example:

```
"married": false
```

2.5 Date (ISODate)

MongoDB stores dates as BSON **Date** objects.

ISODate Format

```
ISODate("YYYY-MM-DDTHH:MM:SSZ")
```

Example:

```
"dob": ISODate("2000-10-25T08:00:00Z")
```

Meaning of Components

- **T** separates date and time
- **Z** represents UTC time
- Offsets like **+02:00** or **-05:00** represent time zone differences
 - **+02:00** → Central European Time
 - **-05:00** → Eastern Standard Time
- Without **Z** or offsets → server local time zone

Saving Current Date

Using **new Date()**:

```
"createdAt": new Date()
```

This stores the current server date and time.

2.6 Double (Floating-Point Numbers)

Example:

```
"weight": 70.8
```

Any number containing a decimal point is treated as **Double**.

2.7 Null

Represents an empty or missing value.

Example:

```
"kids": null
```

2.8 Array

Used for multiple values of the same data type.

Example:

```
"hobbies": ["music", "sports"]
```

2.9 Object (Embedded Document)

Allows nested structure similar to JavaScript objects.

Example:

```
"address": {  
  "street": "MG Road",  
  "city": "Delhi",  
  "zip": 110001  
}
```

2.10 Regular Expression (Not typically used)

Used for pattern matching during queries.

Example:

```
"name": { "$regex": "^A" }
```

2.11 Timestamp

- Similar to date but stored as numeric seconds-based values.
 - Machine-readable, not human-readable.
 - Automatically used by MongoDB internal replication logs.
-

3. ISODate Detailed Format and Time Zones

ISODate example:

```
ISODate("2024-10-18T18:30:00Z")
```

Variants:

UTC Time:

```
"2024-10-18T18:30:00Z"
```

Central European Time (UTC +2):

```
"2024-10-18T18:30:00+02:00"
```

Eastern Standard Time (UTC -5):

```
"2024-10-18T18:30:00-05:00"
```

Local Server Time (no timezone passed):

```
ISODate("2024-10-18T18:30:00")
```

4. Practical Implementation in MongoDB Shell

4.1 Switching Database

```
use school
```

4.2 Show Collections

```
show collections
```

4.3 Creating a New Collection

```
db.createCollection("personal")
```

5. Inserting Documents with All Data Types

Example Document

```
db.personal.insertOne({
  name: "Y Baba",
  age: 25,
  married: false,
  dob: ISODate("2000-10-25T08:00:00Z"),
  weight: 70.5,
  kids: null,
  hobbies: ["music", "sports"],
  address: {
    street: "MG Road",
    city: "Delhi",
    zip: 110001
  }
})
```

Output:

```
acknowledged: true
insertedId: ObjectId("...")
```

6. Inserting Another Document (Modified Values)

```
db.personal.insertOne({
  name: "Akshay Kumar",
  age: 32,
  married: true,
  dob: ISODate("1995-02-15T10:00:00Z"),
  weight: 75.2,
  kids: 2,
  hobbies: ["travel", "books"],
  address: {
```

```
    street: "Park Street",  
    city: "Mumbai",  
    zip: 400001  
  }  
})
```

7. Inserting a Document with Current Date

```
db.personal.insertOne({  
  name: "Mohan Kumar",  
  dob: new Date()  
})
```

8. Viewing All Documents

```
db.personal.find()
```

9. Important Recommendation for Using Dates

Even though a date can be stored as a string such as "2024-10-20", this is discouraged. MongoDB date functions do **not work** on string dates.

Always use:

- `ISODate()`
- or `new Date()`

for proper date handling.
